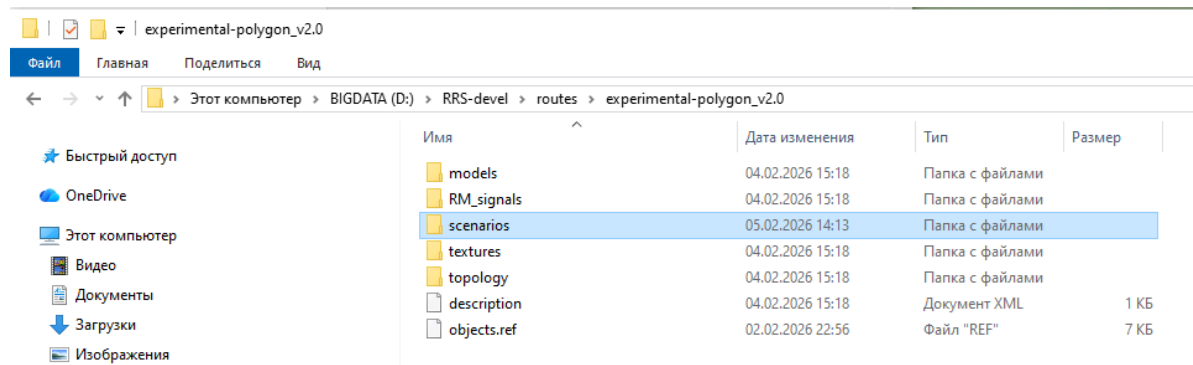


## 2. Структура сценария. Простейшие функции

### 2.1. Простейший сценарий. Установка игрового времени и расположения поезда игрока

Любой сценарий пишется для конкретного маршрута в игре. Поэтому, естественным образом, все сценарии для конкретного маршрута должны располагаться в папке scenarios, расположенной в папке с маршрутом. Например, в состав игры входит маршрут "Испытательный полигон". Папка scenarios уже есть там

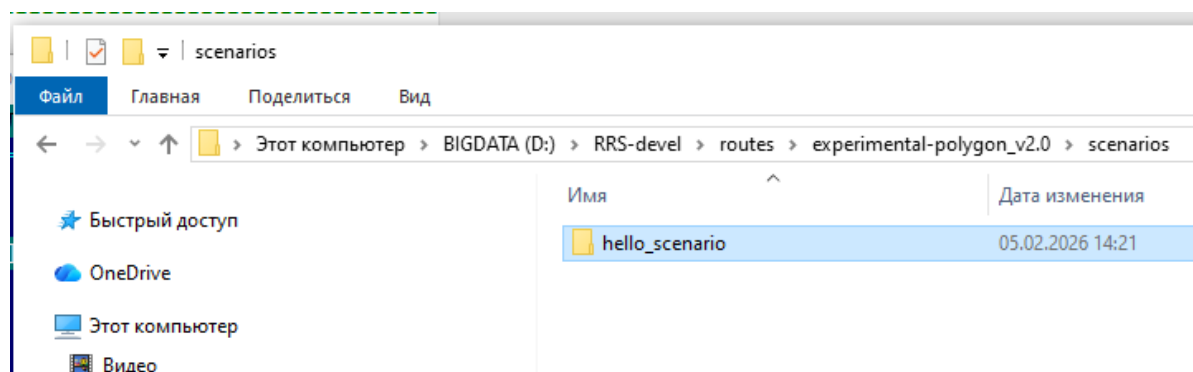


Если в том маршруте, для которого вы ведете разработку сценария, такой папки нет - создайте её!

В данной папке располагаются папки сценариев. Сценарий - это папка, содержащая файлы скриптов, файл описания, а в дальнейшем и звуковые файлы для озвучивания сценариев также будут располагаться там.

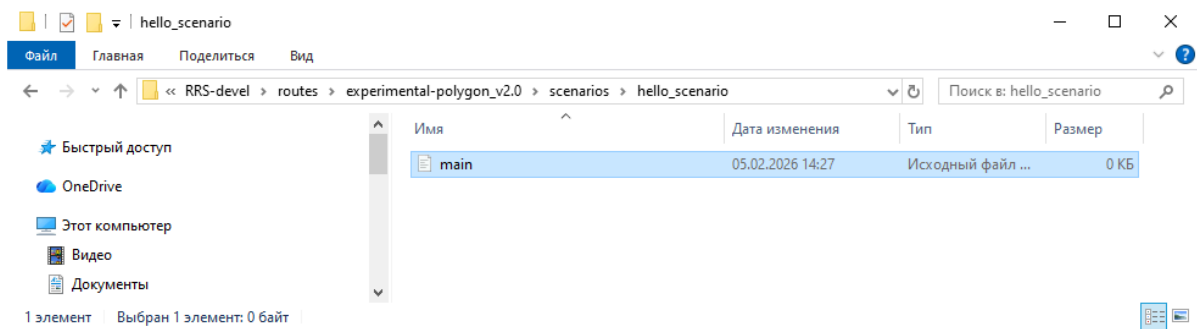
Давайте создадим свой первый сценарий в RRS! Для примера будем использовать стандартный маршрут "Испытательный полигон".

Создадим в папке scenarios папку с именем сценария. Дадим ей имя hello\_scenario



**ВНИМАНИЕ!!!** Избегайте использования в именах папок и файлов сценариев национальных алфавитов и пробельных символов! Несмотря на то, что такие имена поддерживаются как ОС Windows, так и, по-видимому, движком Lua симулятора, не исключены проблемы при поиске и исполнении сценариев игрой! Только латиница, цифры, тире и подчеркивания.

Внутри этой папки создадим файл с именем main.lua



**ВНИМАНИЕ!!!** Файл с именем main.lua обязан присутствовать в сценарии! Это так называемый корневой или стартовый скрипт, с которого начинается исполнение сценария. Игра ожидает наличие этого файла! Расширение \*.lua также является обязательным!

Теперь мы можем написать простейший сценарий. Начнем с того, что зададим время, с которого начнется игра при запуске сценария. Пишем в файле main.lua

```
setTime("17:30")
```

Обратите внимание на синтаксис - время мы указываем в кавычках, так как, с точки зрения игры, астрономическое время это строка. Игра сама преобразует строку в игровое время.

Далее, нам нужно задать поезд, которым мы будем играть. Сделаем это, написав такой код скрипта

```
my_train = TrainData.new()
my_train.name = "ВЛ60пк"
my_train.config = "v160pk-1543"
my_train.traj = "track_a_p1a"
my_train.coord = 1690.0
my_train.dir = 1

setTrain(my_train)
```

Давайте разберем эти "заклинания". my\_train - это структура, которая хранит информацию о поезде и его начальном положении. Вы можете дать ей любое имя, но со следующими ограничениями

- имя содержит ТОЛЬКО латиницу!
- имя не должно начинаться с цифры! Цифры далее - разрешены
- пробелы - не допускаются, тире - не допускаются. Используйте вместо них символ подчеркивания "\_"

Первая строка

```
my_train = TrainData.new()
```

создает в памяти игры структуру описания вашего поезда. TrainData - стандартное имя этой структуры. new() - оператор, который её создает

Далее

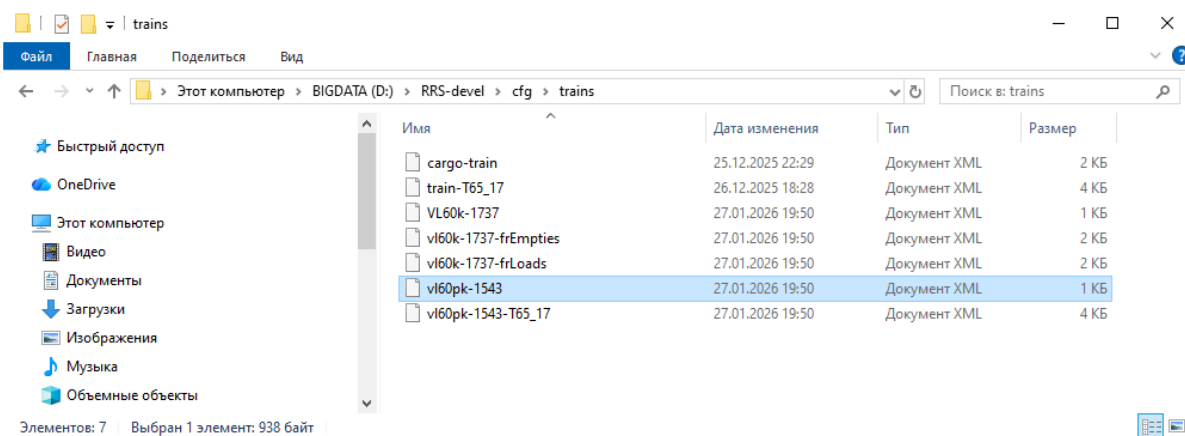
```
my_train.name = "ВЛ60пк"
```

Здесь мы задаем имя нашего поезда. Это строка, поэтому она указана в кавычках! Это имя в дальнейшем используется для идентификации нашего поезда в различных операциях, выполняемых в сценариях.

**ВНИМАНИЕ!!!** Игра допускает здесь применение кириллических имен, как и в некоторых других случаях, которые мы оговорим позднее.

```
my_train.config = "vl60pk-1543"
```

Задаем имя конфига поезда. Эти конфиги лежат в игре в папке `cfg/trains`. Указанный конфиг - это электровоз ВЛ60пк резервом, вот он

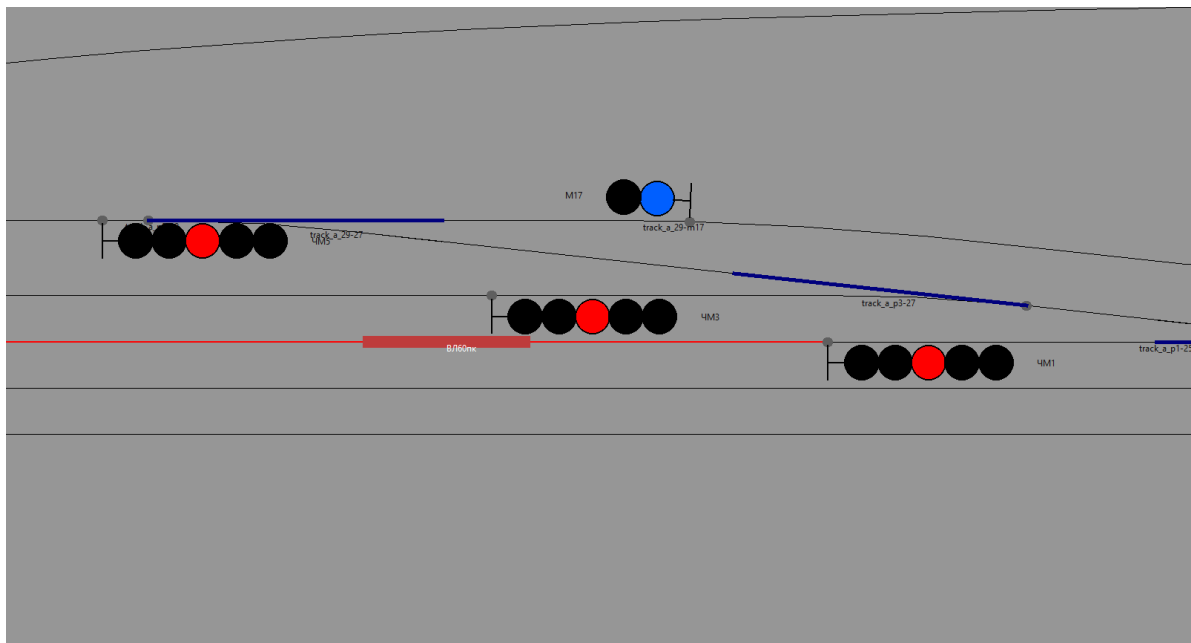


В игру будет загружен поезд, описанный данным конфигом.

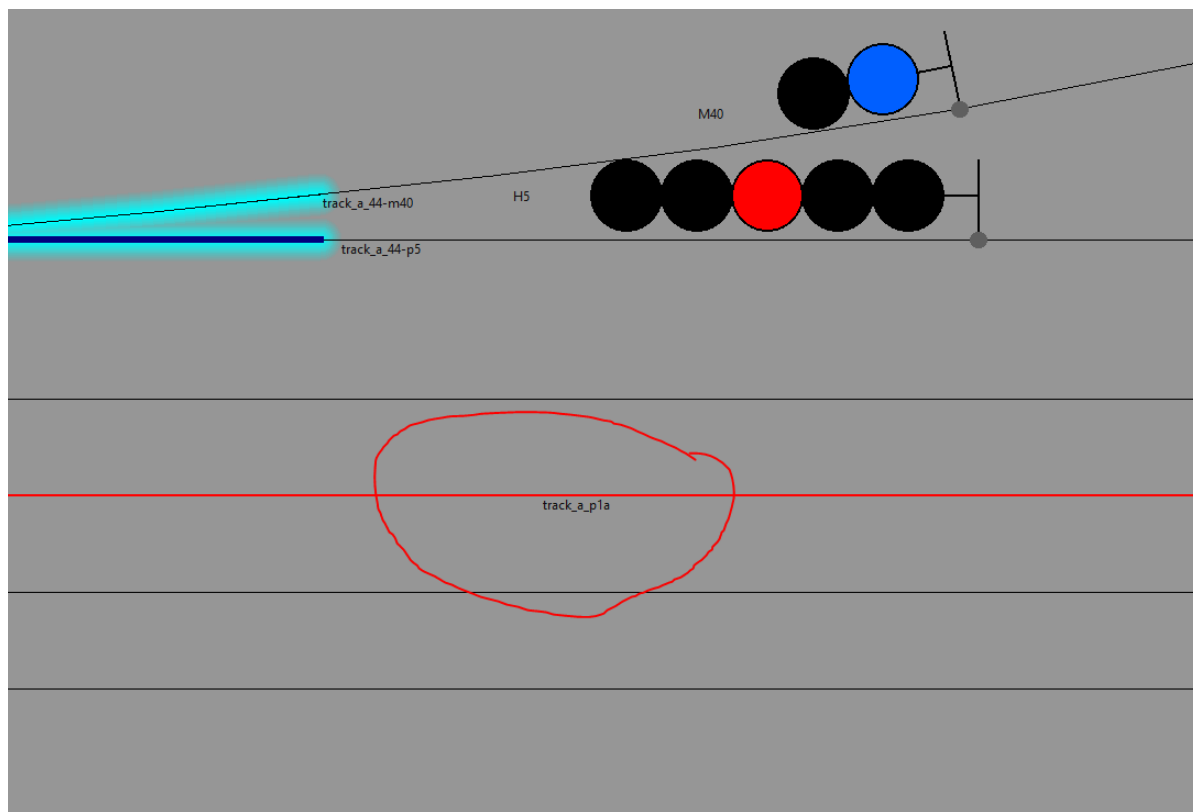
Идем дальше

```
my_train.traj = "track_a_p1a"
```

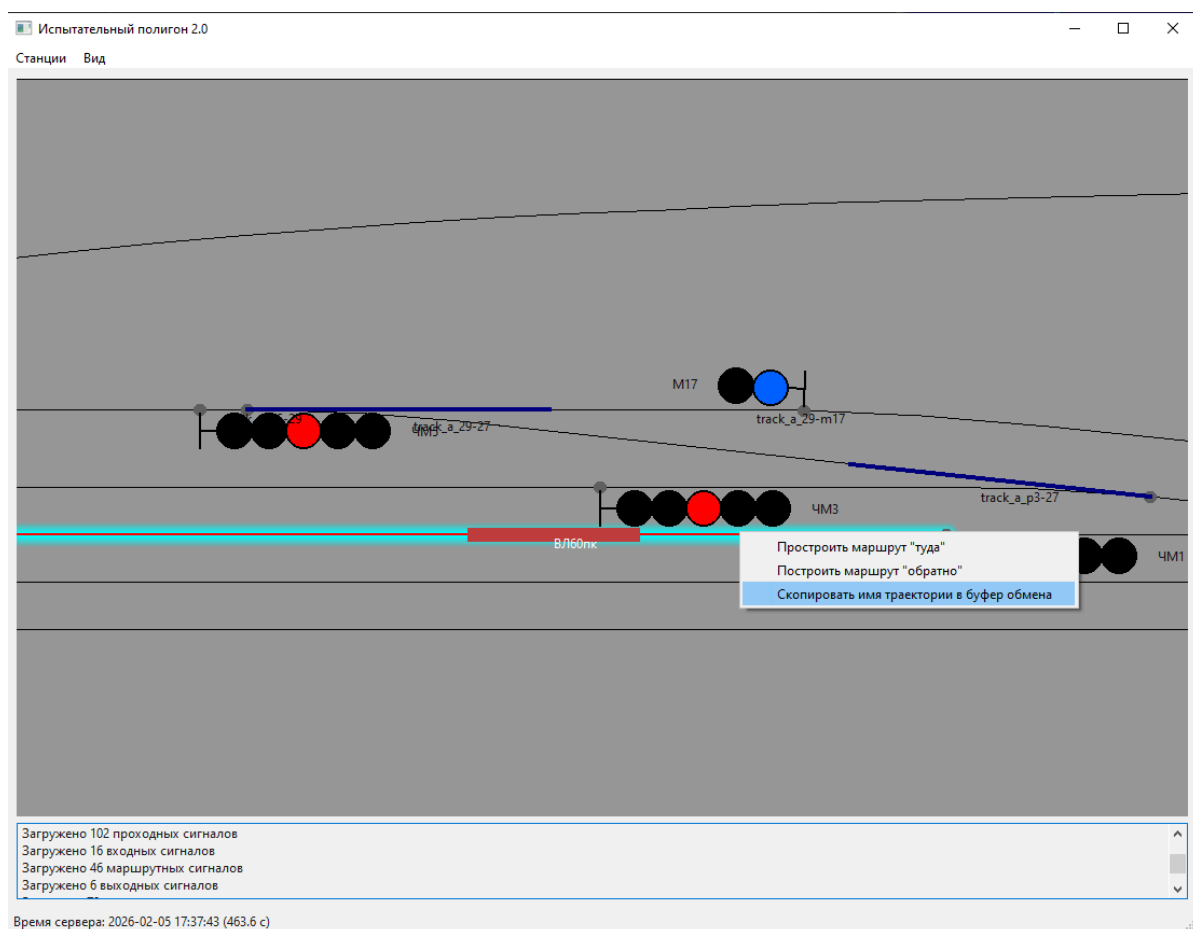
Указываем имя траектории, на которой стоит поезд. Что такое траектория в RRS? Это непрерывный участок пути, разделенный стрелками или изолированными стыками, имеющий собственное имя. В данном случае - это путь 1А на Станции А испытательного полигона. В карте это выглядит так



Как узнать имя траектории? Ну, во-первых, если вы запустите игру в свободном режиме, без сценария, поставив любой поезд куда угодно, вы можете вывести в карте имена всех траекторий через меню "Вид -> Показывать имена траекторий". И тогда вы увидите имя интересующей вас траектории, где-то в её середине. Вот так



Но ползать по карте неудобно, искать имя долго. Есть другой способ получить имя интересующей траектории - выбрать ее, наведя курсор, и вызвать меню правой кнопкой

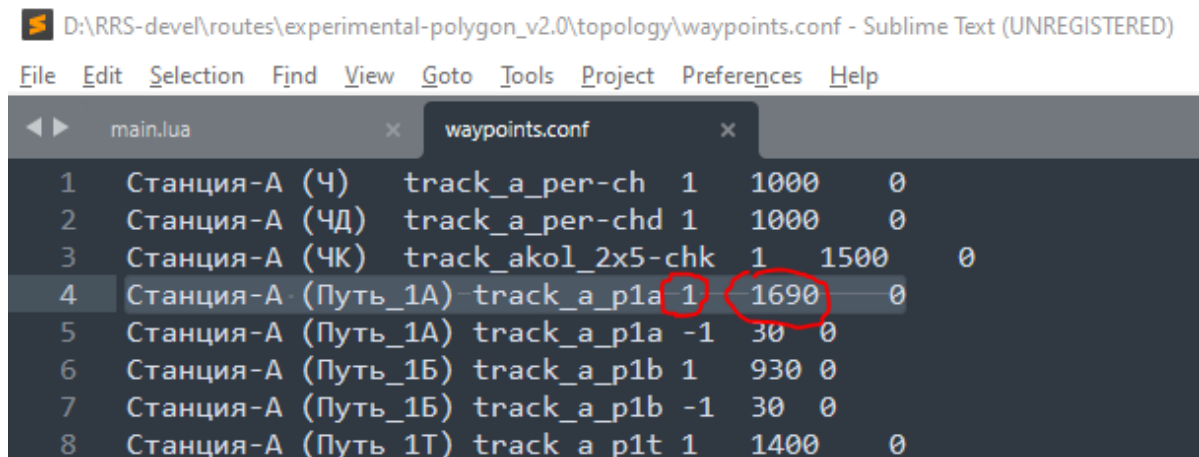


Имя нужной вам траектории будет скопировано в буфер обмена, и в текст сценария вы его можете просто вставить (Ctrl + V / Shift + Insert)

Следующий параметр

```
my_train.coord = 1690.0
```

Координата головы поезда относительно начала траектории, в метрах. Этот параметр можно приблизительно оценить по карте. Другой способ, посмотреть конфиг `topology/waypoints.conf` расположенный в папке с маршрутом. Обычно он включает некоторые траектории на станциях, которые можно использовать в качестве точек старта из лаунчера. Внутри этого файла



перечислены так называемые "путевые точки". Каждая строка включает в себя строку с названием станции и именем пути или выходного светофора с этого пути, имя траектории, направление (1 - в данном случае), и координатой вдоль траектории. Последний параметр

```
my_train.dir = 1
```

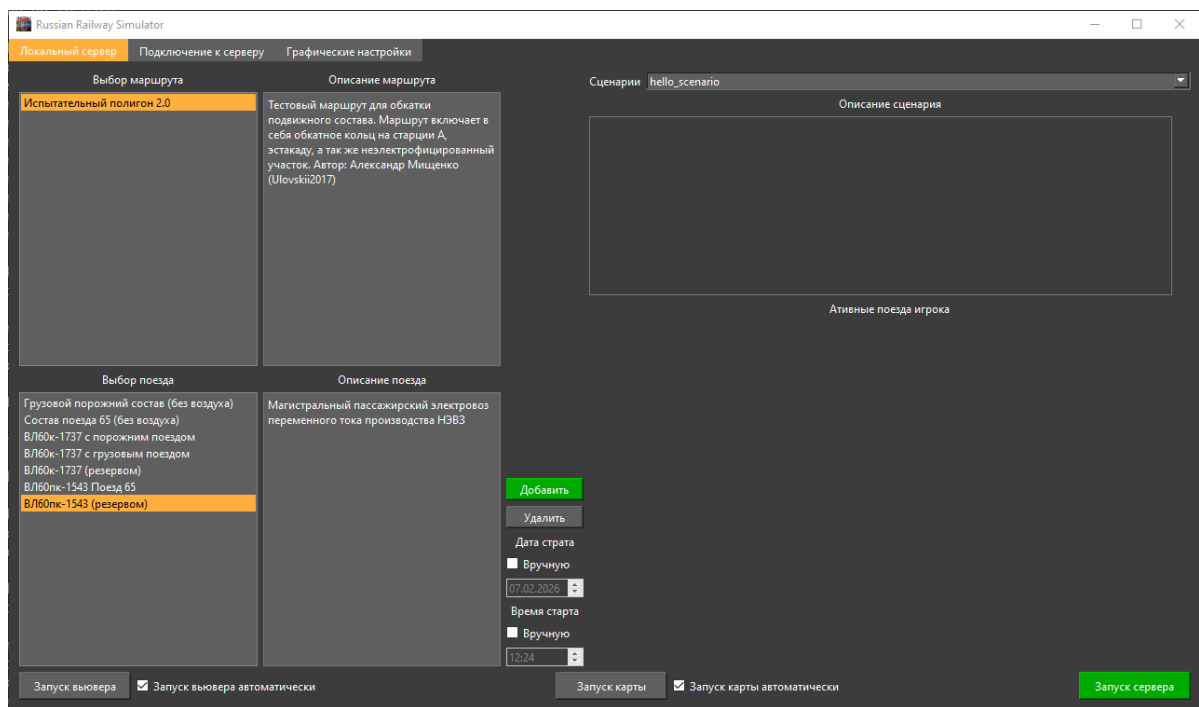
направление головы поезда. В старых терминах маршрутов ZDS 1 - "туда", -1 - "обратно". В терминах RRS - 1 - от начала траектории к концу, -1 - от конца траектории к началу.

Таким образом мы закончили описание поезда и следующей командой устанавливаем поезд в игру

```
setTrain(my_train)
```

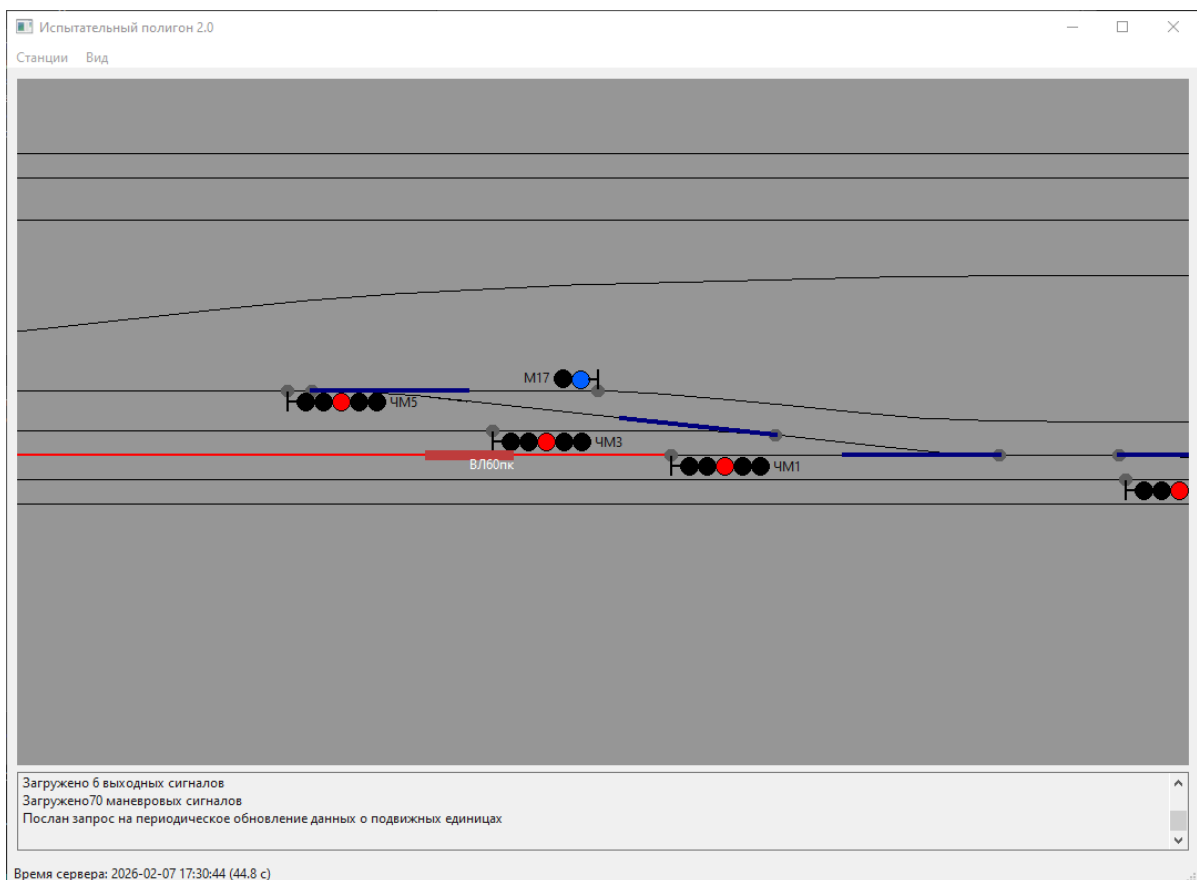
Системная функция `setTrain()` принимает на вход структуру описания поезда.

Мы написали простейший игровой сценарий. Проверим его работу. Запускаем лаунчер игры (`bin/launcher.exe`). Выбираем маршрут и в выпадающем списке "Сценарии" видим наш `hello_scenario`. Выбираем его.

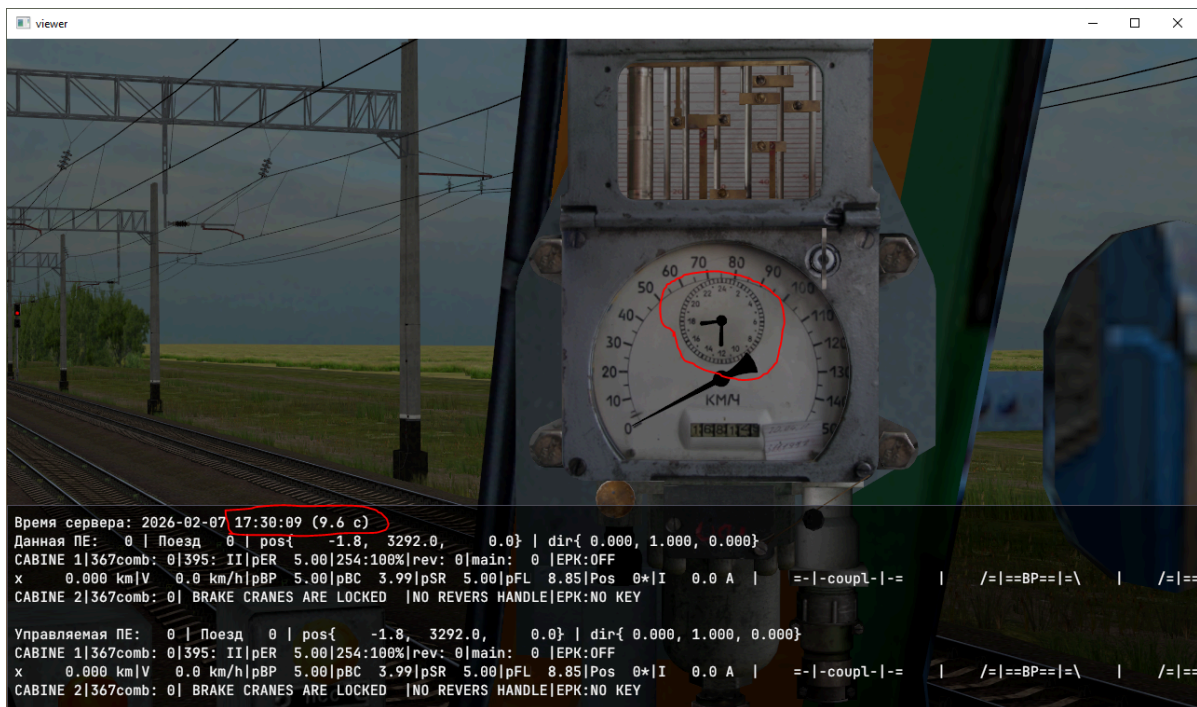


Жмем кнопку "Запуск сервера". Игра запустится, и мы увидим наш поезд, стоящий там, где мы задали в сценарии.





На игровых часах - заданное время, 17:30.



Перед нами простейший сценарий RRS. Но толку от этого сценария довольно мало, давайте теперь попробуем сделать что-то по-настоящему существенное, чего нельзя автоматически выполнить в свободной поездке.

## 2.2. Построение маршрута отправления. Понятие временного триггера

В реальности поезда отправляются по графику, в определенное графикам время. Дежурный по железнодорожной станции или поездной диспетчер *строят* маршрут отправления, переводя нужные стрелки и открывая сигналы.

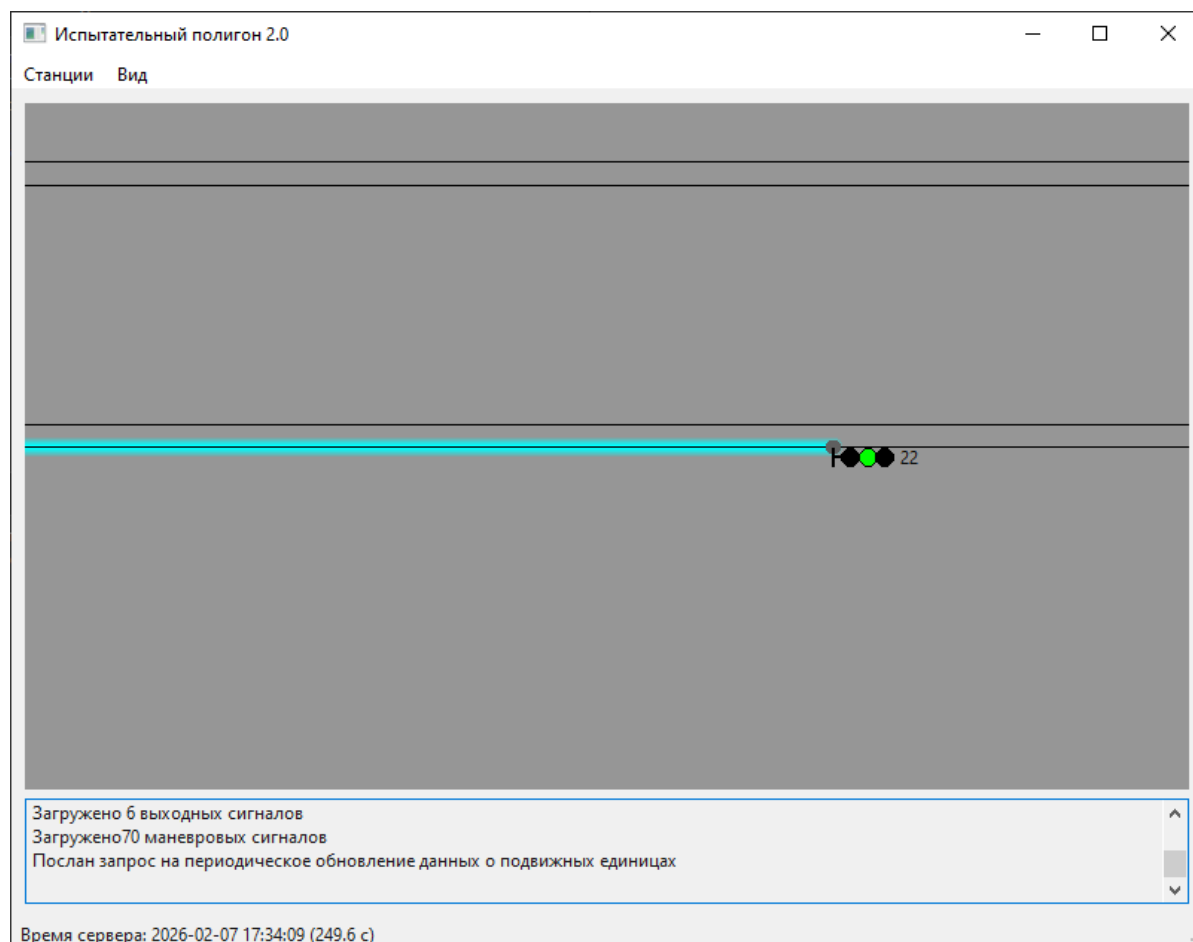
Сценарии RRS позволяют задать такие действия по времени. Для этого предназначены так называемые *временные триггеры* - команда сценария, позволяющая задать произвольное действие, которое выполнится в заданное время. Команда выглядит так

```
setTimeTrigger("время срабатывания", имя_функции_действия)
```

В тексте нашего сценария напишем, например, так

```
setTimeTrigger("17:31", actionBuildTrainRoute("track_a_p1a", "track_a-b_nd-22", 1))
```

Это означает, что в 17:31 по игровому времени будет построен поездной маршрут от траектории, где находится наш поезд ("track\_a\_p1a"), до траектории на первом участке удаления от станции А, то есть до вот этой траектории, которая упирается в первый проходной светофор 22 на перегоне.



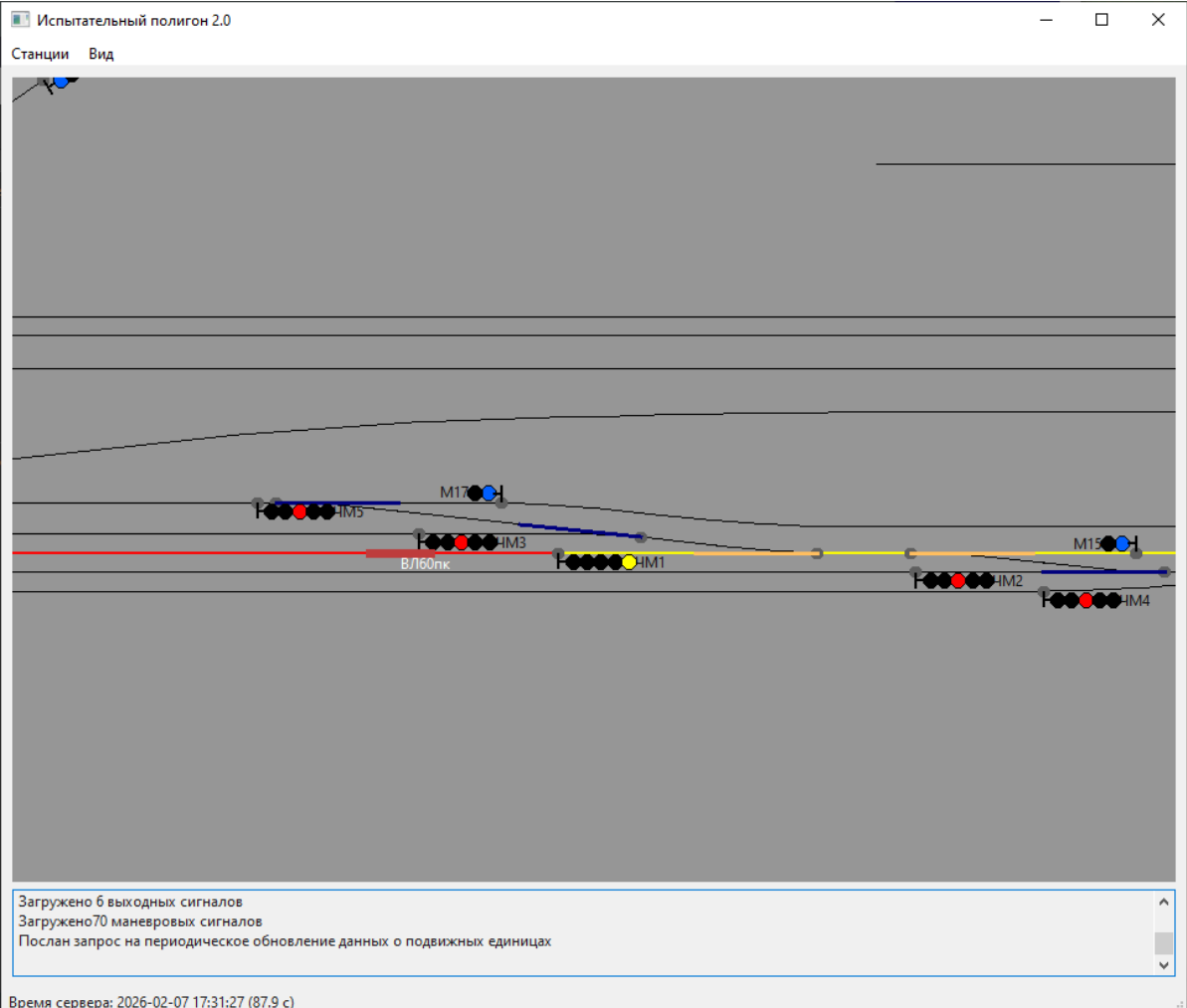
За построение маршрута отвечает отдельная функция

```
actionBuildTrainRoute("track_a_p1a", "track_a-b_nd-22", 1)
```



Это стандартная action-функция, или "функция-действие", которая строит поездной маршрут от одной траектории до другой в заданном направлении, в нашем случае в направлении "туда" (1).

Попробуем, что у нас вышло. Перезапускаем игру с нашим сценарием. Ровно в 17:31 открывается маршрутный сигнал ЧМ1, и на карте отображается построенный маршрут отправления



Можно отправляться!

Временной триггер можно задать иначе - установить время относительно времени старта игры. Сделать это можно, указав "+" перед значением времени

```
setTimeTrigger("+00:01", actionBuildTrainRoute("track_a_p1a", "track_a-b_nd-22", 1))
```

Это означает, что маршрут будет построен через 1 минуту после начала игры. Вы можете поэкспериментировать, указав теперь произвольное время в setTime. Маршрут будет построен ровно через минуту от указанного вами времени начала игры.

Для большинства типовых действий игрой предусмотрены стандартные action-функции, полный список которых приведен в [Приложении 2](#).

Таким образом, вот полный текст нашего первого сценария для RRS.

```
setTime("17:30")

my_train = TrainData.new()
my_train.name = "ВЛ60пк"
my_train.config = "v160pk-1543"
my_train.traj = "track_a_p1a"
my_train.coord = 1690.0
my_train.dir = 1

setTrain(my_train)

setTimeTrigger("17:31", actionBuildTrainRoute("track_a_p1a", "track_a-b_nd-22", 1))
```

Согласитесь, выглядит как книга заклинаний. Чтобы сделать этот текст более понятным как для себя, так и для других, можно использовать комментарии. Ведь Lua - полноценный язык программирования и допускает использования комментариев.

В Lua есть два вида комментариев: однострочные и многострочные. Однострочный комментарий начинается с двух идущих подряд тире "--". Например,

```
-- Задаем время начала игры
setTime("17:30")

-- Описываем поезд игрока
my_train = TrainData.new()
my_train.name = "ВЛ60пк"
my_train.config = "v160pk-1543"
my_train.traj = "track_a_p1a"
my_train.coord = 1690.0
my_train.dir = 1

-- Устанавливаем поезд игрока
setTrain(my_train)

-- Строим маршрут отправления в 17:31 с пути 1А станции А
setTimeTrigger("17:31", actionBuildTrainRoute("track_a_p1a", "track_a-b_nd-22", 1))
```

Многострочный комментарий начинается с последовательности символов "--[[\" и оканчивается последовательностью \"]]", например

```
--[[

Мой первый сценарий
для Russian Railway Simulator

]]

-- Задаем время начала игры
setTime("17:30")

-- Описываем поезд игрока
my_train = TrainData.new()
my_train.name = "ВЛ60пк"
my_train.config = "v160pk-1543"
my_train.traj = "track_a_p1a"
my_train.coord = 1690.0
my_train.dir = 1

-- Устанавливаем поезд игрока
setTrain(my_train)

-- Строим маршрут отправления в 17:31 с пути 1А станции А
setTimeTrigger("17:31", actionBuildTrainRoute("track_a_p1a", "track_a-b_nd-22",
1))
```

Крайне рекомендуем использовать комментарии! Это поможет и вам разобраться в том, что вы написали, и другим сценаристам вникнуть в суть происходящего в вашем сценарии. Кроме того, комментарии могут использоваться для того, чтобы сделать неактивной команду или несколько команд сценария, превратив их временно в комментарии (например при отладке, если что-то пошло не так).

## 2.3. Если что-то пошло не так

Может так получиться, что вы напишете сценарий, например вот так

```
--[[

Мой первый сценарий
для Russian Railway Simulator

]]

-- Задаем время начала игры
setTime("17:30")

-- Описываем поезд игрока
my_train = TraiData.new()
my_train.name = "ВЛ60пк"
my_train.config = "v160pk-1543"
my_train.traj = "track_a_p1a"
my_train.coord = 1690.0
```

```

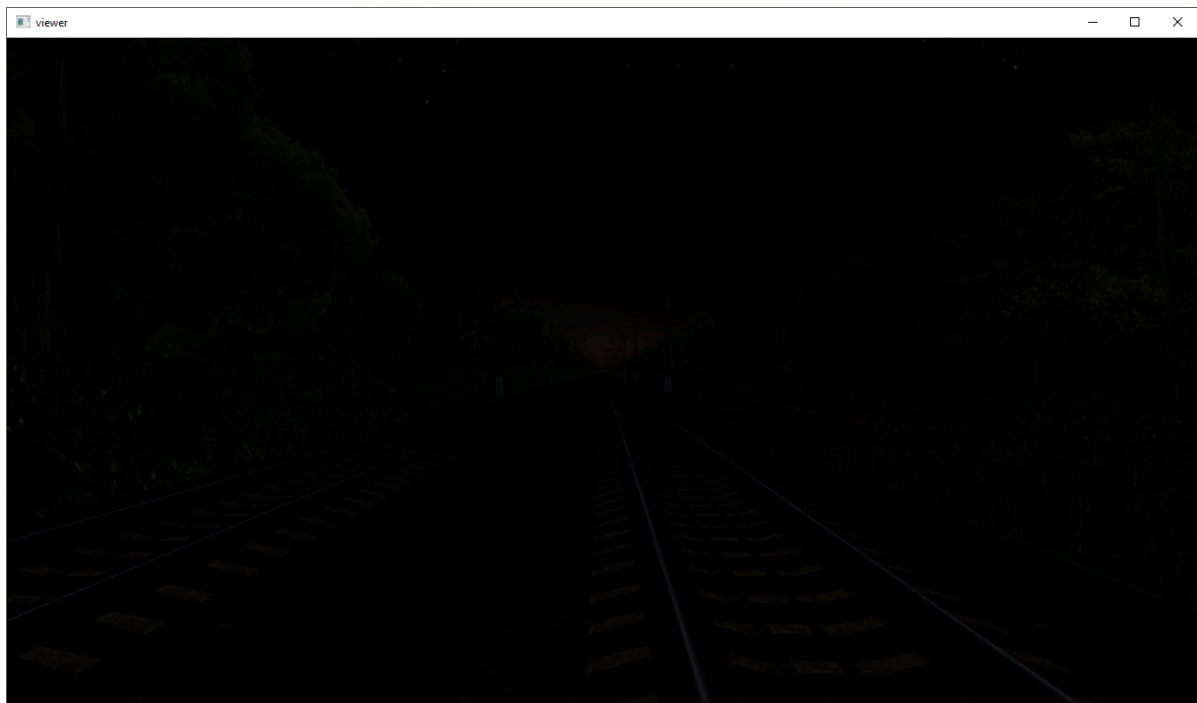
my_train.dir = 1

-- Устанавливаем поезд игрока
setTrain(my_train)

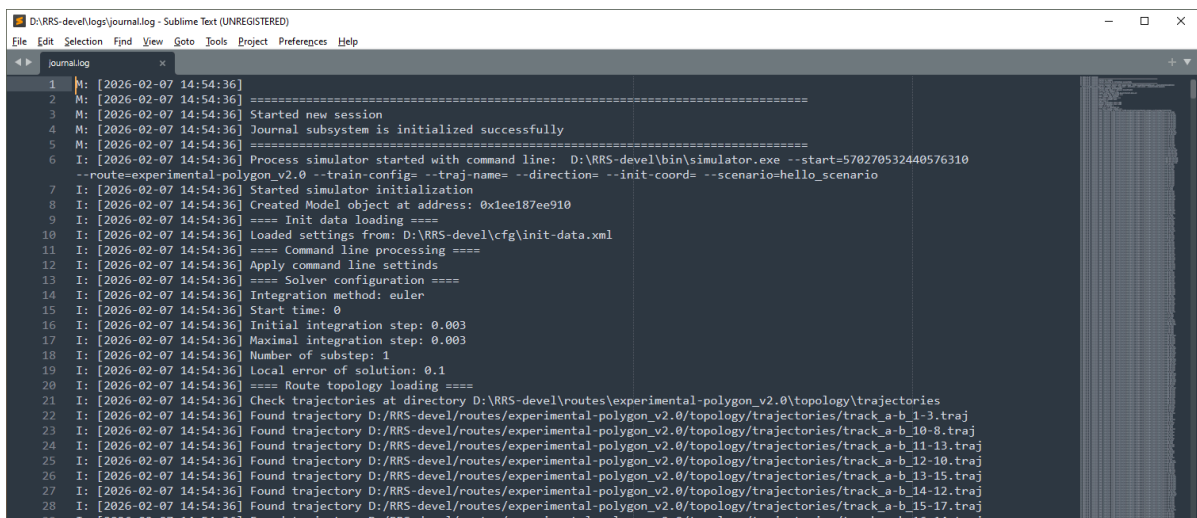
-- Строим маршрут отправления в 17:31 с пути 1А станции А
setTimeTrigger("17:31", actionBuildTrainRoute("track_a_pla", "track_a-b_nd-22",
1))

```

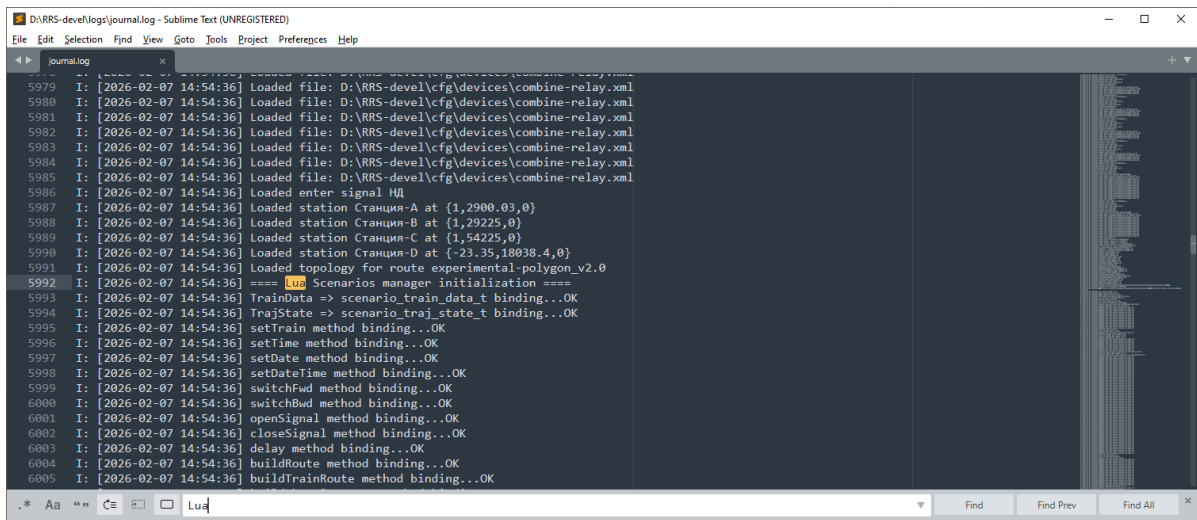
а вместо поезда на станции увидите это



Что делать? То, что вы видите, явный признак того, что где-то в скрипте допущена одна или несколько ошибок. В этом случае вам поможет журнал запуска игры, расположенный в файле logs/journal.log. Откройте его текстовым редактором

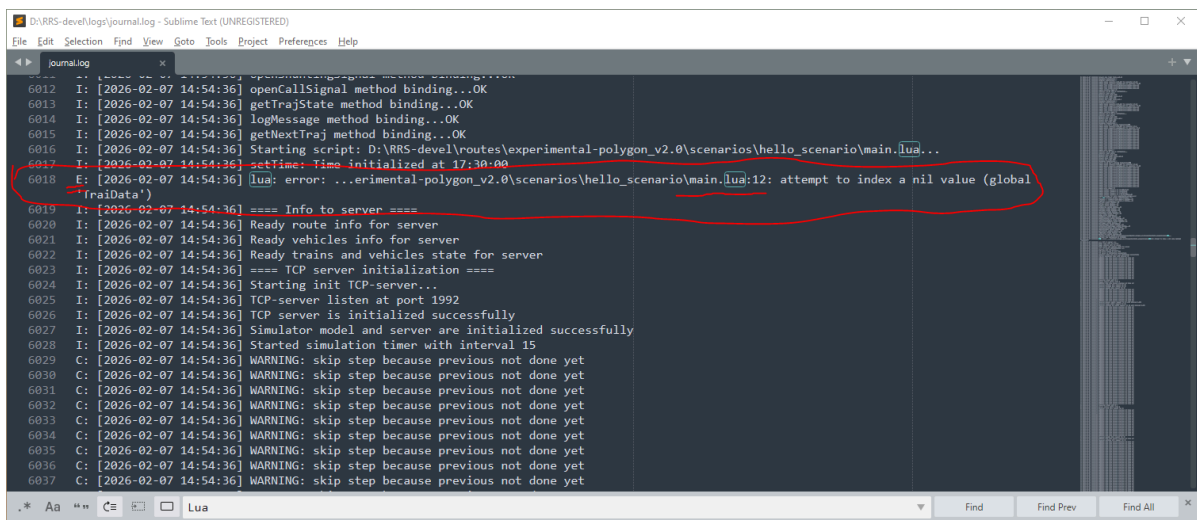


Здесь отражаются очень многие действия, происходящие при старте и работе игры. Тут много всего, поэтому воспользуемся поиском по тексту (Ctrl + F) по ключевому слову "Lua"



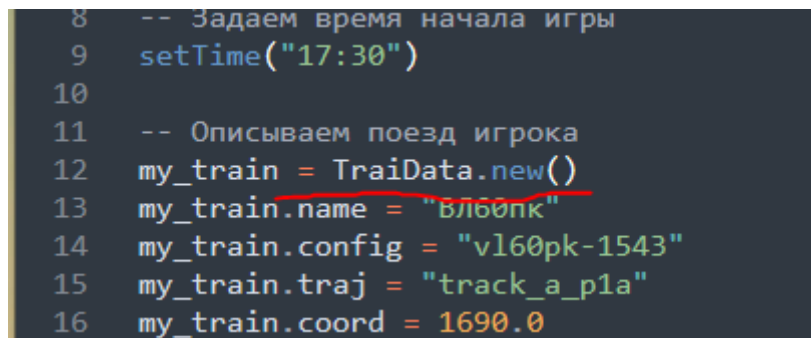
```
5979 I: [2026-02-07 14:54:36] Loaded file: D:\RRS-devel\cfg\devices\combine-relay.xml
5980 I: [2026-02-07 14:54:36] Loaded file: D:\RRS-devel\cfg\devices\combine-relay.xml
5981 I: [2026-02-07 14:54:36] Loaded file: D:\RRS-devel\cfg\devices\combine-relay.xml
5982 I: [2026-02-07 14:54:36] Loaded file: D:\RRS-devel\cfg\devices\combine-relay.xml
5983 I: [2026-02-07 14:54:36] Loaded file: D:\RRS-devel\cfg\devices\combine-relay.xml
5984 I: [2026-02-07 14:54:36] Loaded file: D:\RRS-devel\cfg\devices\combine-relay.xml
5985 I: [2026-02-07 14:54:36] Loaded file: D:\RRS-devel\cfg\devices\combine-relay.xml
5986 I: [2026-02-07 14:54:36] Loaded enter signal HQ
5987 I: [2026-02-07 14:54:36] Loaded station Станция-A at {1,2900,0,0}
5988 I: [2026-02-07 14:54:36] Loaded station Станция-B at {1,20225,0}
5989 I: [2026-02-07 14:54:36] Loaded station Станция-C at {1,54225,0}
5990 I: [2026-02-07 14:54:36] Loaded station Станция-D at {-23,35,18038,4,0}
5991 I: [2026-02-07 14:54:36] Loaded topology for route experimental-polygon_v2.0
5992 I: [2026-02-07 14:54:36] ==== Lua Scenarios manager initialization ====
5993 I: [2026-02-07 14:54:36] TrainData => scenario_train_data_t binding...OK
5994 I: [2026-02-07 14:54:36] TrajState => scenario_traj_state_t binding...OK
5995 I: [2026-02-07 14:54:36] setTrain method binding...OK
5996 I: [2026-02-07 14:54:36] setTime method binding...OK
5997 I: [2026-02-07 14:54:36] setDate method binding...OK
5998 I: [2026-02-07 14:54:36] setDateTime method binding...OK
5999 I: [2026-02-07 14:54:36] switchFwd method binding...OK
6000 I: [2026-02-07 14:54:36] switchBwd method binding...OK
6001 I: [2026-02-07 14:54:36] openSignal method binding...OK
6002 I: [2026-02-07 14:54:36] closeSignal method binding...OK
6003 I: [2026-02-07 14:54:36] delay method binding...OK
6004 I: [2026-02-07 14:54:36] buildRoute method binding...OK
6005 I: [2026-02-07 14:54:36] buildTrainRoute method binding...OK
6012 I: [2026-02-07 14:54:36] openCallSignal method binding...OK
6013 I: [2026-02-07 14:54:36] getTrajState method binding...OK
6014 I: [2026-02-07 14:54:36] logMessage method binding...OK
6015 I: [2026-02-07 14:54:36] getNextTraj method binding...OK
6016 I: [2026-02-07 14:54:36] Starting script: D:\RRS-devel\routes\experimental-polygon_v2.0\scenarios\hello_scenario\main.lua...
6017 I: [2026-02-07 14:54:36] setTime: time initialized at 17:30:00
6018 E: [2026-02-07 14:54:36] Lua: error: ...erimental-polygon_v2.0\scenarios\hello_scenario\main.lua:12: attempt to index a nil value (global TraiData)
6019 I: [2026-02-07 14:54:36] ==== Info to server ====
6020 I: [2026-02-07 14:54:36] Ready route info for server
6021 I: [2026-02-07 14:54:36] Ready vehicles info for server
6022 I: [2026-02-07 14:54:36] Ready trains and vehicles state for server
6023 I: [2026-02-07 14:54:36] ==== TCP server initialization ====
6024 I: [2026-02-07 14:54:36] Starting init TCP-server...
6025 I: [2026-02-07 14:54:36] TCP-server listen at port 1992
6026 I: [2026-02-07 14:54:36] TCP server is initialized successfully
6027 I: [2026-02-07 14:54:36] Simulator model and server are initialized successfully
6028 I: [2026-02-07 14:54:36] Started simulation timer with interval 15
6029 C: [2026-02-07 14:54:36] WARNING: skip step because previous not done yet
6030 C: [2026-02-07 14:54:36] WARNING: skip step because previous not done yet
6031 C: [2026-02-07 14:54:36] WARNING: skip step because previous not done yet
6032 C: [2026-02-07 14:54:36] WARNING: skip step because previous not done yet
6033 C: [2026-02-07 14:54:36] WARNING: skip step because previous not done yet
6034 C: [2026-02-07 14:54:36] WARNING: skip step because previous not done yet
6035 C: [2026-02-07 14:54:36] WARNING: skip step because previous not done yet
6036 C: [2026-02-07 14:54:36] WARNING: skip step because previous not done yet
6037 C: [2026-02-07 14:54:36] WARNING: skip step because previous not done yet
```

Мы нашли место, где выводятся сообщения об обработке сценария игрой. Листаем чуть вниз и находим сообщение об ошибке



```
6012 I: [2026-02-07 14:54:36] openCallSignal method binding...OK
6013 I: [2026-02-07 14:54:36] getTrajState method binding...OK
6014 I: [2026-02-07 14:54:36] logMessage method binding...OK
6015 I: [2026-02-07 14:54:36] getNextTraj method binding...OK
6016 I: [2026-02-07 14:54:36] Starting script: D:\RRS-devel\routes\experimental-polygon_v2.0\scenarios\hello_scenario\main.lua...
6017 I: [2026-02-07 14:54:36] setTime: time initialized at 17:30:00
6018 E: [2026-02-07 14:54:36] Lua: error: ...erimental-polygon_v2.0\scenarios\hello_scenario\main.lua:12: attempt to index a nil value (global TraiData)
6019 I: [2026-02-07 14:54:36] ==== Info to server ====
6020 I: [2026-02-07 14:54:36] Ready route info for server
6021 I: [2026-02-07 14:54:36] Ready vehicles info for server
6022 I: [2026-02-07 14:54:36] Ready trains and vehicles state for server
6023 I: [2026-02-07 14:54:36] ==== TCP server initialization ====
6024 I: [2026-02-07 14:54:36] Starting init TCP-server...
6025 I: [2026-02-07 14:54:36] TCP-server listen at port 1992
6026 I: [2026-02-07 14:54:36] TCP server is initialized successfully
6027 I: [2026-02-07 14:54:36] Simulator model and server are initialized successfully
6028 I: [2026-02-07 14:54:36] Started simulation timer with interval 15
6029 C: [2026-02-07 14:54:36] WARNING: skip step because previous not done yet
6030 C: [2026-02-07 14:54:36] WARNING: skip step because previous not done yet
6031 C: [2026-02-07 14:54:36] WARNING: skip step because previous not done yet
6032 C: [2026-02-07 14:54:36] WARNING: skip step because previous not done yet
6033 C: [2026-02-07 14:54:36] WARNING: skip step because previous not done yet
6034 C: [2026-02-07 14:54:36] WARNING: skip step because previous not done yet
6035 C: [2026-02-07 14:54:36] WARNING: skip step because previous not done yet
6036 C: [2026-02-07 14:54:36] WARNING: skip step because previous not done yet
6037 C: [2026-02-07 14:54:36] WARNING: skip step because previous not done yet
```

Здесь написано, что в 12 строке файла main.lua упомянуто несуществующее имя TraiData. Смотрим на двенадцатую строку и действительно видим, что мы допустили опечатку



```
8 -- Задаем время начала игры
9 setTime("17:30")
10
11 -- Описываем поезд игрока
12 my_train = TraiData.new()
13 my_train.name = "ВЛ60пк"
14 my_train.config = "vl60pk-1543"
15 my_train.traj = "track_a_p1a"
16 my_train.coord = 1690.0
```

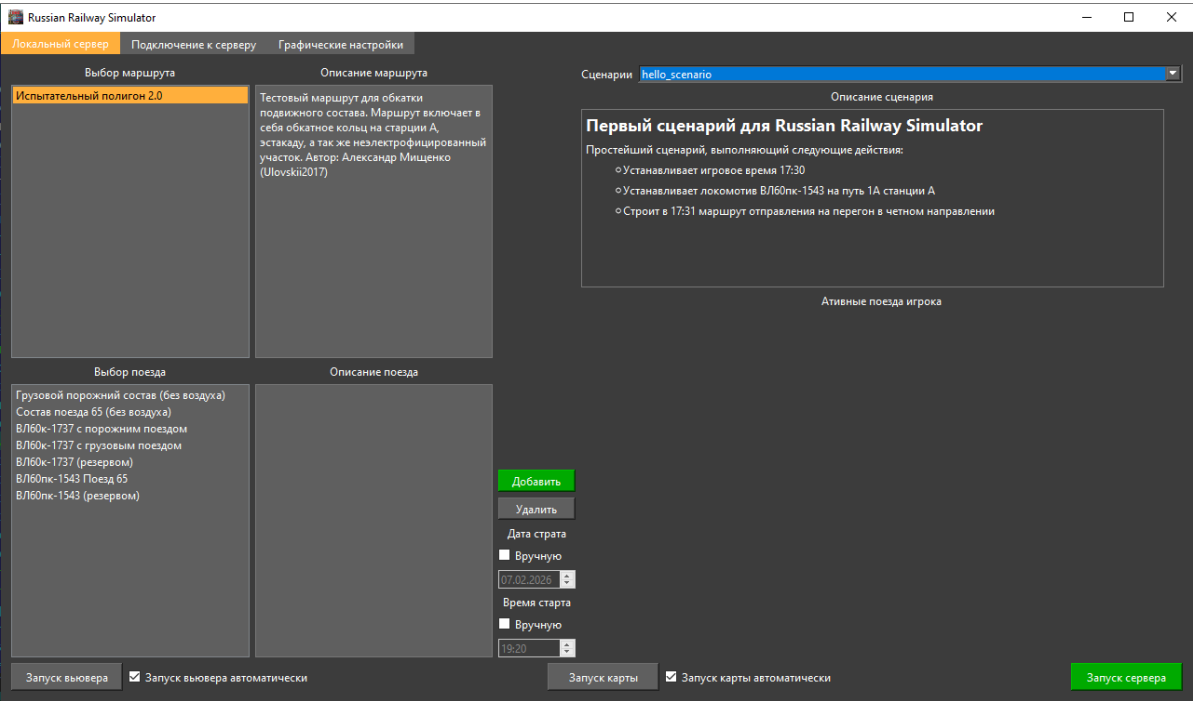
пропустив букву "n" в названии структуры описания поезда TrainData. После исправления этой ошибки игра нормально запустится.

## 2.4. Файл README.md описания сценария

Согласитесь, название папки со сценарием, которое отображает лаунчер - малоинформативно. Поэтому предусмотрена возможность создавать файл описания сценария.

Этот файл должен иметь название README.md (Это обязательно, с точностью до регистра и расширения!!!).

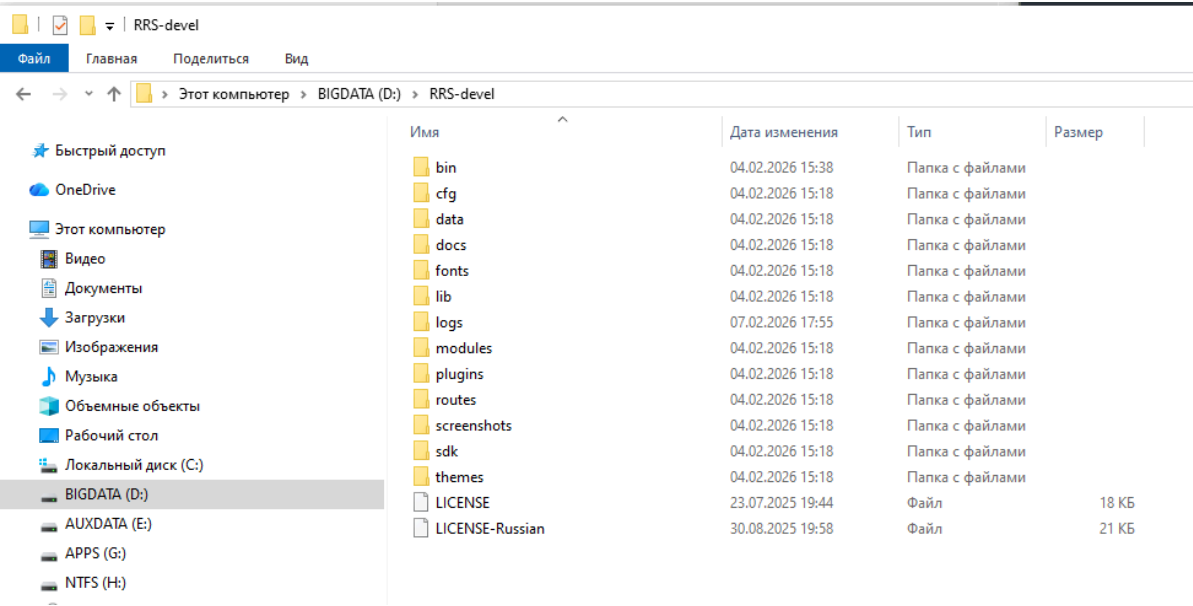
Этот файл текстовый, может содержать произвольный текст в кодировке UTF-8. Он допускает использование разметки Markdown. Если таковой файл присутствует в папке с вашим сценарием, то его содержимое выводится в окно описания сценария при выборе сценария в лаунчере.



## 2.5. Сценарий, как дополнение к игре. Структура папок

Мы написали простейший сценарий, и продолжим изучение системы сценариев RRS в дальнейшем. Однако, раз речь в данной главе зашла о структуре сценария, то следует сказать, что сценарий - такое же дополнение симулятора, как подвижной состав или маршрут. Потому грамотно разработанный сценарий должен соответствовать структуре дополнения для RRS.

Вспомним структуру каталогов игры



Маршруты располагаются в каталоге routes. Значит, ваш сценарий, в готовом для распространения виде, например для маршрута "Испытательный полигон", должен повторять данную структуру каталогов



```
my_scenario/-
  |-routes/-
    |-experimental-polygon_v2.0/-
      |-scenarios-
        |-my_scenario1
        |-my_scenario2
```

Таким образом, ваш проект сценариев для данного маршрута может содержать несколько папок с разными сценариями.

Например, вам может понадобиться использовать сценарии, которые используют конфигурации поездов, которых нет у игрока. Вы можете заранее сформировать эти поезда, написав их конфиги, и положить в комплект сценария, разместив папке `cfg/trains`, например

```
my_scenario/-
  |-cfg/-
    |-trains/-
      |-train104.xml
      |-train105.xml
  |-routes/-
    |-experimental-polygon_v2.0/-
      |-scenarios-
        |-my_scenario1
        |-my_scenario2
```

Здесь `train104.xml` и `train105.xml` - конфиги поездов, которые вы составили сами и используете в сценарии.

Можно пойти дальше, например, ваш поезд `train104.xml` использует нестандартный конфиг для электровоза ВЛ60пк, которого нет в игре. Вы можете, учтя структуру каталогов игры, поставить такой конфиг вместе со сценарием. Не буду подробно описывать это здесь, но такая возможность существует, и её реализация требует более глубокого понимания структуры игры, которое можно почерпнуть из изучения документации на сайте <https://rusrailsim.ru>